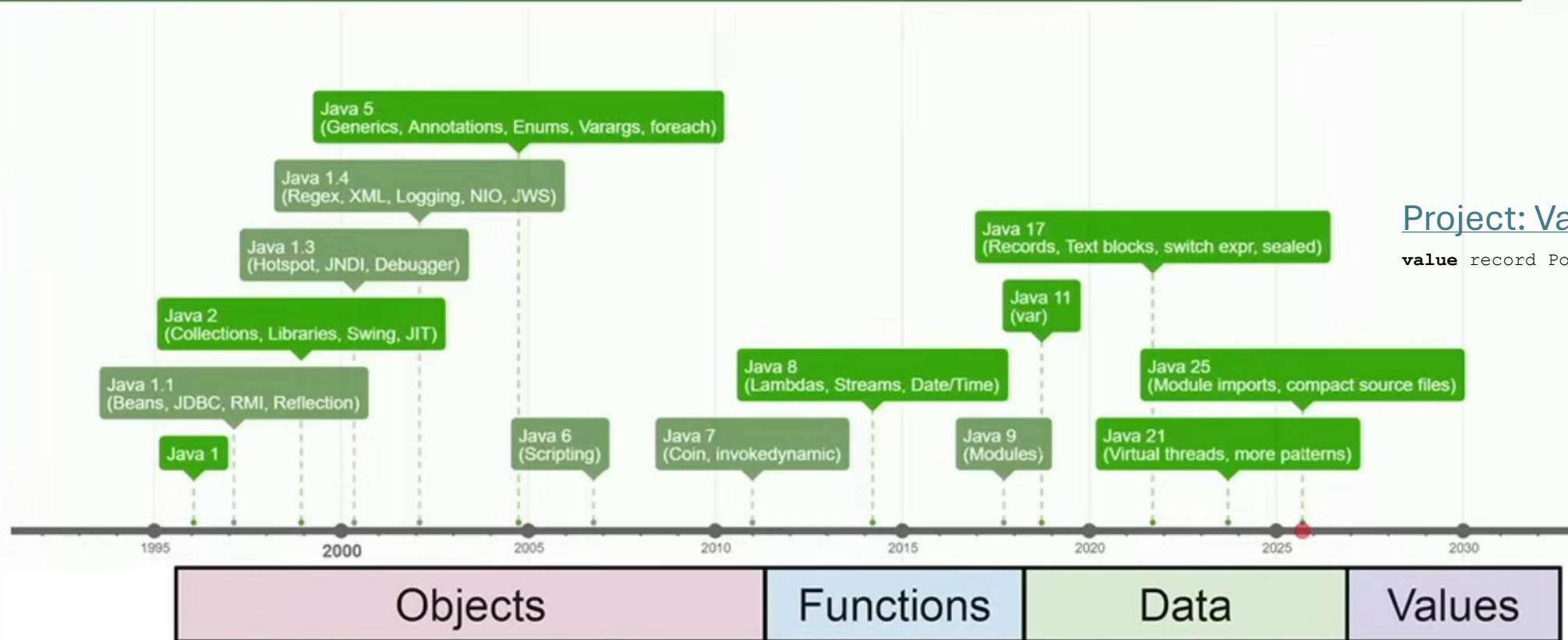


Data-oriented Programming

Pattern Matching und Exhaustiveness

Eras of Java



Project: Valhalla

```
value record Point(int x, int y) {}
```

Statement vs Expression

Ein **Statement** ist eine vollständige Anweisung, die **ausgeführt wird**, aber **keinen Wert zurückgibt**.

```
int number = 42;    // Zuweisung , kein Rückgabewert
IO.println(number); // Methodenaufruf, kein Rückgabewert

switch (day) { // kein Rückgabewert
    case MONDAY -> IO.println("Start der Woche");
}
```

Eine **Expression** ist ein Ausdruck, der **einen Wert zurückgibt**.

```
"Hallo".length(); // returns 5
1+2; // returns 3

String message = switch (day) { // returns a String
    case MONDAY -> "Start der Woche";
    case FRIDAY -> "Fast Wochenende";
    default -> "Normaler Tag";
}
```

Sealed

sealed interface Shape **permits** Circle, Rectangle, Triangle {}

```
record Circle(double radius) implements Shape {}  
record Rectangle(double width, double height) implements Shape {}  
record Triangle(double base, double height) implements Shape {}
```

```
class ShapeDemo {  
    void main() {  
        Shape shape = new Rectangle(4, 5);  
  
        double area = switch (shape) {  
            case Circle c -> Math.PI * c.radius() * c.radius();  
            case Rectangle r -> r.width() * r.height();  
            case Triangle t -> (t.base() * t.height()) / 2;  
        };  
  
        IO.println("Fläche: " + area);  
    }  
}
```

Neue Formen lassen sich mit Anpassung der permits-Liste hinzufügen. Dies führt dazu, dass der Compiler überall auf **Exhaustivness** prüft, wo switch mit einem Shape-Interface verwendet wird, d. h., wenn ein case fehlt, gibt es einen Compilerfehler.

sealed interface

Nur die angegebenen Klassen (Circle, Rectangle, Triangle) dürfen Shape implementieren.

record (oder class)

Datenträger für die Shapes.

switch mit Pattern Matching

Automatisches Typ-Casting und Zuweisung.

sealed

Der Compiler prüft, dass alle erlaubten Subtypen als case vorliegen.

Sealed vs Record

Mit record und sealed lassen sich in der Programmiersprache Eigenschaften modellieren:

record (auch mit class möglich)

`record` Rectangle(double width, double height) implements Shape {}

→ Rectangle ist (besteht aus) width **UND** height.

sealed (mit record, class und interface möglich)

`sealed interface` Shape `permits` Circle, Rectangle {}

→ Shape ist ein Circle **ODER** Rectangle.

Exhaustiveness

Das Konzept der **Exhaustiveness** (Vollständigkeit) bedeutet, dass ein switch-Ausdruck alle möglichen Fälle für einen Typ (oder Enum) durch cases abdecken muss.

Mit **sealed** ist **Exhaustiveness möglich**, weil:

- Die Typ-Hierarchie geschlossen ist (nur die in permits genannten Subtypen sind erlaubt).
- Vorteil: Es ist kein default-case nötig (aber weiterhin erlaubt), weil der Compiler bei einem sealed Typ die Vollständigkeit garantieren kann. Das macht den Code typsicher und **robuster gegenüber Änderungen**.

Switch

Statement

Expression

Label

fall-through

```
switch (day) {  
  case MONDAY: ...;  
  break;  
  case TUESDAY: ...;  
  break;  
}
```

```
switch (day) {  
  case null: ...;  
  break;  
  case MONDAY: ...;  
  break;  
  case TUESDAY: ...;  
  break;  
  default: ...; break;  
}
```

```
var result = switch (day) {  
  case MONDAY: ...; yield 6;  
  case TUESDAY: ...; yield 7;  
  default: ...; yield -1;  
}
```

Arrow

No fall-through

```
switch (day) {  
  case MONDAY -> ...;  
  case TUESDAY -> ...;  
}
```

```
switch (day) {  
  case null -> ...;  
  case MONDAY -> ...;  
  case TUESDAY -> ...;  
  default -> ...;  
}
```

```
var result = switch (day) {  
  case MONDAY -> ...;  
  case TUESDAY -> ...;  
  default -> ...;  
}
```

BEST PRACTICE

Not exhaustive

Exhaustive
case null, or Patterns

Exhaustive

Exhaustiveness: Switch in Java:

Aus historischen Gründen, gibt es in Java zwei Implementierungen von switch:

1. Die Alte, die nicht auf Exhaustiveness prüft
→ bei byte, short, char, int, enum und String.
2. Die Neue („enhanced switch“), die auf Exhaustiveness prüft.
→ bei boolean, long, float, double, und sobald ein case mit null oder Pattern Matching vorhanden ist.

Type Pattern Matching mit instanceof

Pattern Matching erlaubt es, ein Objekt **auf einen Typ zu prüfen** und **gleichzeitig eine Variable für diesen Typ zu binden**.

```
if (obj instanceof String) { // Klassisch mit instanceof und cast  
    String shape = (String) obj;  
    IO.println(shape.toUpperCase());  
}
```

```
if (obj instanceof String shape) { // Type Pattern Matching  
    IO.println(shape.toUpperCase());  
}
```

Type Pattern Matching mit switch

In dem folgenden Beispiel wird das Pattern (Circle c, Rectangle r) direkt im switch verwendet.

→ Kein instanceof und kein explizites Casting notwendig.

→ Der Compiler wählt automatisch den passenden case.

In Kombination mit sealed kann der Compiler hier auch Exhaustiveness prüfen.

→ Kein default-case notwendig.

```
sealed interface Shape permits Circle, Rectangle {}
```

```
record Circle(double r) implements Shape {}
```

```
record Rectangle(double w, double h) implements Shape {}
```

```
double area(Shape shape) {  
    return switch (shape) {  
        case Circle c -> Math.PI * c.r() * c.r();  
        case Rectangle r -> r.w() * r.h();  
    };  
}
```

Type Pattern Matching mit switch und Guards

```
sealed interface Shape permits Circle, Rectangle {}
```

```
record Circle(double radius, Color color) implements Shape {}  
record Rectangle(double width, double height) implements Shape {}
```

```
class SwitchWithGuards {  
    void main() {  
        Shape shape = new Rectangle(4, 4);  
  
        String description = switch (shape) {  
            case Circle c when c.radius() > 10 -> "Großer Kreis";  
            case Circle c -> "Kleiner Kreis";  
            case Rectangle r when r.width() == r.height() -> "Quadrat";  
            case Rectangle r -> "Rechteck";  
        };  
  
        IO.println(description);  
    }  
}
```

Guards sind eine zusätzliche Bedingung für Pattern Matching. Guards werden nach **case** verwendet und durch das Schlüsselwort **when** eingeleitet.

Ein case mit Guard wird nur ausgeführt, wenn der **Typ** passt **und** die **Bedingung** erfüllt ist.

Ein **case mit einem Guard zählt niemals zur Exhaustiveness**. Für Exhaustiveness ist daher ein weiteres case ohne Guard für den jeweiligen Typ notwendig.

Record Pattern Matching mit Switch und Guards

```
sealed interface Shape permits Circle, Rectangle {}
```

```
record Circle(double radius, Color color) implements Shape {}  
record Rectangle(double width, double height) implements Shape {}
```

```
class SwitchWithGuards {  
    void main() {  
        Shape shape = new Rectangle(4, 4);  
  
        String description = switch (shape) {  
            case Circle(double r, Color _) when r > 10 -> "Großer Kreis";  
            case Circle(var r, var _) -> "Kleiner Kreis";  
            case Rectangle(double w, double h) when w == h -> "Quadrat";  
            case Rectangle(var w, var h) -> "Rechteck";  
        };  
  
        IO.println(description);  
    }  
}
```

Mit Record Pattern Matching lassen sich Attribute direkt aus einem record extrahieren.

Ein **Underscore** `_` wird verwendet, um zu signalisieren, dass ein Attribut für den case nicht benötigt bzw. benutzt wird.

Hier ist **ausnahmsweise** auch **var** bei Parametern **erlaubt** (sonst nur bei lokalen Variablen).

Vorteil: Direkter Zugriff auf einzelne Attribute ohne Methodenaufruf.

Nachteil: Diese Variante funktioniert nur mit records (nicht bei normalen Klassen).

Polymorphie (objektorientiert)

Fokus: Verhalten in Objekte kapseln

Polymorphie ist ein Kernprinzip:

Man definiert eine gemeinsame Schnittstelle oder Basisklasse. Jede konkrete Klasse implementiert ihr eigenes Verhalten.

Der Methodenaufruf erfolgt zur Laufzeit anhand der jeweiligen Objektinstanz (d. h., der Compiler ruft die Implementation der Klasse des jeweiligen Objekts auf).

Es lassen sich problemlos neue Shape-Typen hinzufügen, ohne dass etwas am existierenden Code verändert werden muss.

```
interface {  
    double area();  
}
```

```
record Circle(double r) implements Shape {  
    public double area() { return Math.PI * r * r; }  
}
```

```
record Rectangle(double w, double h) implements Shape {  
    public double area() { return w * h; }  
}
```

```
double calc(Shape shape) {  
    return shape.area(); // Polymorpher Aufruf, d. h., Circle::area() oder Rectangle::area()  
}
```

Pattern Matching (datenorientiert)

```
sealed interface Shape permits Circle, Rectangle {}
```

```
double area(Shape shape) { // Operation 1
    return switch (shape) {
        case Circle c -> Math.PI * c.r() * c.r();
        case Rectangle r -> r.w() * r.h();
    };
}
```

```
double outline(Shape shape) { // Operation 2
    return switch (shape) {
        case Circle c -> 2 * Math.PI * c.r();
        case Rectangle r -> 2 * (r.w() + r.h());
    };
}
```

// Weitere Operationen möglich...

Fokus: Datenstrukturen + zentrale Logik

Verhalten wird nicht in Objekte gekapselt, sondern in Funktionen, die über Daten entscheiden.

Mit Pattern Matching + switch lassen sich zentral alle Varianten behandeln.

Die Logik (für alle cases) befindet sich an einem Ort (Operation 1, Operation 2, Operation n).

FYI: Bei Polymorphie wird die Logik hingegen auf viele Klassen verteilt.

Der Compiler kann zusätzlich auf Exhaustiveness prüfen, sodass kein Fall aus Versehen vergessen wird (z. B. wenn eine neue Klasse wie Triangle hinzukommt).

Polymorphie vs Pattern Matching

Aspekt	Polymorphie (objektorientiert)	Pattern Matching (datenorientiert)
Verhalten (Logik)	In Klassen (Typ)	Zentral in Funktion (Operation)
Erweiterbarkeit	Offen für neue Typen	Offen für neue Operationen
Typsicherheit	Dynamisch zur Laufzeit	Statisch geprüft (Exhaustiveness)
Performance	Keine Kompileroptimierung möglich	Kompileroptimierung möglich
Lesbarkeit	Verteilt über Klassen	Gebündelt in einer Operation.

Wann Polymorphie verwenden? (objektorientiert)

- Viele Typen (z. B. viele Shapes) und wenige gemeinsame Operationen (area(), outline()).
- Zukünftig eher neue Typen hinzukommen.
- Nachteil: Neue Methoden im Interface bedeuten Anpassungen in allen Implementierungen.
- Empfehlenswert: viele Typen, wenige Operationen.

Wann Pattern Matching verwenden? (datenorientiert)

- Wenn die Typen eher Datenstrukturen sind und die Logik zentral gehalten werden soll.
- Wenige Typen existieren (z. B. drei Shapes), aber viele verschiedene Operationen (z. B. 20 Berechnungen).
- Zukünftig wahrscheinlich neue Operationen hinzukommen.
- Nachteil: Neuer Typ im sealed Interface bedeutet Anpassungen in allen switch-Anweisungen.
- Empfehlenswert: wenige Typen, viele Operationen.

Anwendungsfälle: Polymorphie

(objektorientiert)

- **Strategie-Pattern**

Beispiel: Verschiedene Algorithmen für Sortierung (QuickSort, MergeSort) implementieren ein gemeinsames Interface SortStrategy.

- **Domain-Modelle mit komplexem Verhalten**

Beispiel: In einer Banking-App haben Account, SavingsAccount, CreditAccount jeweils eigene Logik für calculateInterest().

- **Plug-in-Systeme**

Neue Typen (Plug-ins) kommen oft hinzu, aber die Schnittstelle bleibt gleich / stabil.

- **GUI-Frameworks**

Beispiel: Component-Interface mit Methoden wie draw().

Jede konkrete Komponente (Button, TextField, Checkbox) implementiert ihr eigenes Zeichnen.

Anwendungsfälle: Pattern Matching

(datenorientiert)

- **Serialisierung/Deserialisierung**

Beispiel: JSON/XML-Konvertierung für verschiedene Datenstrukturen, zentral in einer Methode.

- **Business-Logik mit wenigen Typen, aber vielen Regeln**

Beispiel: Versicherungsberechnung für 3 Vertragsarten, aber 20 verschiedene Berechnungsregeln.

- **Protokoll-Handling**

Beispiel: Netzwerkprotokolle mit wenigen Nachrichtentypen, aber viele Operationen wie Validierung, Logging, Transformation.

- **Parsing und Baumstrukturen**

Beispiel: Compiler oder Interpreter, die über einen sealed AST-Typ schalten (Expression, Statement, Literal) und viele Operationen wie evaluate(), optimize(), prettyPrint() implementieren.